

Sử dụng OAuth 2.0 cho ứng dụng máy chủ web

Tài liệu này giải thích cách các ứng dụng máy chủ web sử dụng Thư viện ứng dụng API của Google hoặc các điểm cuối OAuth 2.0 của Google để triển khai hoạt động uỷ quyền OAuth 2.0 nhằm truy cập vào các API của Google.

OAuth 2.0 cho phép người dùng chia sẻ dữ liệu cụ thể với một ứng dụng mà vẫn giữ kín tên người dùng, mật khẩu và các thông tin khác. Ví dụ: một ứng dụng có thể sử dụng OAuth 2.0 để được người dùng cấp quyền lưu trữ tệp trong Google Drive của họ.

Quy trình OAuth 2.0 này dành riêng cho việc uỷ quyền cho người dùng. Thư mục này được thiết kế cho những ứng dụng có thể lưu trữ thông tin bí mật và duy trì trạng thái. Một ứng dụng máy chủ web được uỷ quyền đúng cách có thể truy cập vào một API trong khi người dùng tương tác với ứng dụng hoặc sau khi người dùng rời khỏi ứng dụng.

Các ứng dụng máy chủ web cũng thường sử dụng tài khoản dịch vụ

(<https://developers.google.com/identity/protocols/oauth2/service-account?hl=vi>) để uỷ quyền cho các yêu cầu API, đặc biệt là khi gọi API Cloud để truy cập vào dữ liệu dựa trên dự án thay vì dữ liệu dành riêng cho người dùng. Các ứng dụng máy chủ web có thể sử dụng tài khoản dịch vụ cùng với việc uỷ quyền cho người dùng.

Lưu ý: Vì những tác động đến tính bảo mật của việc triển khai chính xác, bạn nên sử dụng các thư viện OAuth 2.0 khi tương tác với các điểm cuối OAuth 2.0 của Google. Bạn nên sử dụng mã đã được gỡ lỗi kỹ lưỡng do người khác cung cấp. Điều này sẽ giúp bạn bảo vệ chính mình và người dùng. Để biết thêm thông tin, hãy xem phần Thư viện ứng dụng (#libraries).

Thư viện ứng dụng

Các ví dụ dành riêng cho ngôn ngữ trên trang này sử dụng Thư viện ứng dụng API của Google (<https://developers.google.com/api-client-library?hl=vi>) để triển khai hoạt động uỷ quyền OAuth 2.0. Để chạy các mẫu mã, trước tiên, bạn phải cài đặt thư viện ứng dụng cho ngôn ngữ của mình.

Khi bạn sử dụng Thư viện ứng dụng API của Google để xử lý quy trình OAuth 2.0 của ứng dụng, thư viện ứng dụng sẽ thực hiện nhiều thao tác mà ứng dụng sẽ cần xử lý theo cách riêng. Ví dụ:

nó xác định thời điểm ứng dụng có thể sử dụng hoặc làm mới mã truy cập đã lưu trữ cũng như thời điểm ứng dụng phải lấy lại sự đồng ý. Thư viện ứng dụng cũng tạo ra các URL chuyển hướng chính xác và giúp triển khai trình xử lý chuyển hướng trao đổi mã ủy quyền để lấy mã truy cập.

Thư viện ứng dụng API của Google cho các ứng dụng phía máy chủ có sẵn cho các ngôn ngữ sau:

- Go (<https://github.com/googleapis/google-api-go-client>)
- Java (<https://github.com/googleapis/google-auth-library-java>)
- .NET (<https://developers.google.com/api-client-library/dotnet?hl=vi>)
- Node.js (<https://github.com/googleapis/google-api-nodejs-client>)
- Dart (<https://dart.dev/googleapis/>)
- PHP (<https://github.com/googleapis/google-api-php-client>)
- Python (<https://github.com/googleapis/google-api-python-client>)
- Ruby (<https://github.com/googleapis/google-api-ruby-client>)

Lưu ý quan trọng: Thư viện ứng dụng Google API cho JavaScript

(<https://developers.google.com/api-client-library/javascript?hl=vi>) và Đăng nhập bằng Google (<https://developers.google.com/identity/gsi/web/guides/overview?hl=vi>) **chỉ** dùng để xử lý OAuth 2.0 trong trình duyệt của người dùng. Nếu bạn muốn sử dụng JavaScript ở phía máy chủ để quản lý các hoạt động tương tác OAuth 2.0 với Google, hãy cân nhắc sử dụng thư viện Node.js (<https://github.com/googleapis/google-api-nodejs-client>) trên nền tảng phụ trợ của bạn.

Điều kiện tiên quyết

Bật API cho dự án của bạn

Mọi ứng dụng gọi API Google đều cần bật các API đó trong API Console.

Cách bật một API cho dự án:

1. Open the API Library (<https://console.developers.google.com/apis/library?hl=vi>) trong Google API Console.
2. If prompted, select a project, or create a new one.
3. API Library liệt kê tất cả những API có sẵn, được nhóm theo nhóm sản phẩm và mức độ phổ biến. Nếu API bạn muốn bật không xuất hiện trong danh sách, hãy sử dụng chức năng tìm kiếm để tìm API đó hoặc nhấp vào **Xem tất cả** trong nhóm sản phẩm mà API đó thuộc về.
4. Chọn API bạn muốn bật, sau đó nhấp vào nút **Bật**.
5. If prompted, enable billing.
6. If prompted, read and accept the API's Terms of Service.

Tạo thông tin xác thực uỷ quyền

Mọi ứng dụng sử dụng OAuth 2.0 để truy cập vào các API của Google đều phải có thông tin xác thực uỷ quyền để xác định ứng dụng cho máy chủ OAuth 2.0 của Google. Các bước sau đây giải thích cách tạo thông tin đăng nhập cho dự án của bạn. Sau đó, các ứng dụng của bạn có thể sử dụng thông tin đăng nhập này để truy cập vào các API mà bạn đã bật cho dự án đó.

1. Go to the Clients page (<https://console.developers.google.com/auth/clients?hl=vi>).
2. Nhấp vào **Tạo ứng dụng**.
3. Chọn loại ứng dụng **Ứng dụng web**.
4. Điền thông tin vào biểu mẫu rồi nhấp vào **Tạo**. Các ứng dụng sử dụng ngôn ngữ và khung như PHP, Java, Python, Ruby và .NET phải chỉ định **URI chuyển hướng** được uỷ quyền. Các URI chuyển hướng là những điểm cuối mà máy chủ OAuth 2.0 có thể gửi phản hồi. Các điểm cuối này phải tuân thủ các quy tắc xác thực của Google ([#uri-validation](#)).

Để kiểm thử, bạn có thể chỉ định các URI tham chiếu đến máy cục bộ, chẳng hạn như `http://localhost:8080`. Xin lưu ý rằng tất cả các ví dụ trong tài liệu này đều sử dụng `http://localhost:8080` làm URI chuyển hướng.

Bạn nên thiết kế các điểm cuối uỷ quyền của ứng dụng (`#protectauthcode`) sao cho ứng dụng của bạn không để lộ mã uỷ quyền cho các tài nguyên khác trên trang.

Sau khi tạo thông tin đăng nhập, hãy tải tệp **client_secret.json** xuống từ API Console. Lưu trữ tệp một cách an toàn ở một vị trí mà chỉ ứng dụng của bạn mới có thể truy cập.

Lưu ý quan trọng: Không lưu trữ tệp **client_secret.json** ở một vị trí có thể truy cập công khai. Ngoài ra, nếu bạn chia sẻ mã nguồn cho ứng dụng của mình (ví dụ: trên GitHub), hãy lưu trữ tệp **client_secret.json** bên ngoài cây nguồn để tránh vô tình chia sẻ thông tin xác thực ứng dụng khách.

Khoá bí mật của ứng dụng chỉ xuất hiện sau khi bạn tạo ứng dụng. Bạn sẽ không thể xem hoặc tải lại mật khẩu ứng dụng khách xuống. [Tìm hiểu thêm](https://support.google.com/cloud/answer/15549257?hl=vi#client-secret-hashing) (https://support.google.com/cloud/answer/15549257?hl=vi#client-secret-hashing).

Xác định phạm vi truy cập

Phạm vi cho phép ứng dụng của bạn chỉ yêu cầu quyền truy cập vào những tài nguyên cần thiết, đồng thời cho phép người dùng kiểm soát mức độ truy cập mà họ cấp cho ứng dụng của bạn. Do đó, có thể có mối quan hệ nghịch đảo giữa số lượng phạm vi được yêu cầu và khả năng nhận được sự đồng ý của người dùng.

Trước khi bắt đầu triển khai quy trình uỷ quyền OAuth 2.0, bạn nên xác định những phạm vi mà ứng dụng của bạn cần có quyền truy cập.

Bạn cũng nên yêu cầu ứng dụng của mình truy cập vào các phạm vi uỷ quyền thông qua quy trình [uỷ quyền từng phần](#) (#incrementalAuth). Trong quy trình này, ứng dụng của bạn sẽ yêu cầu quyền truy cập vào dữ liệu người dùng trong bối cảnh. Phương pháp hay nhất này giúp người dùng dễ dàng hiểu được lý do ứng dụng của bạn cần quyền truy cập mà ứng dụng đang yêu cầu.

Tài liệu [Phạm vi API OAuth 2.0](#) (https://developers.google.com/identity/protocols/oauth2/scopes?hl=vi) chứa danh sách đầy đủ các phạm vi mà bạn có thể dùng để truy cập vào các API của Google.

Nếu ứng dụng công khai của bạn sử dụng các phạm vi cho phép truy cập vào một số dữ liệu người dùng, thì ứng dụng đó phải hoàn tất quy trình xác minh. Nếu thấy thông báo **ứng dụng chưa được xác minh** trên màn hình khi kiểm thử ứng dụng, bạn phải gửi yêu cầu xác minh để xoá thông báo này. Tìm hiểu thêm về [các ứng dụng chưa được xác minh](#) (https://support.google.com/cloud/answer/7454865?hl=vi) và xem câu trả lời cho [các câu hỏi thường gặp về quy trình xác minh ứng dụng](#) (https://support.google.com/cloud/answer/9110914?hl=vi) trong Trung tâm trợ giúp.

Yêu cầu theo ngôn ngữ cụ thể

Để chạy bất kỳ mẫu mã nào trong tài liệu này, bạn cần có Tài khoản Google, quyền truy cập vào Internet và một trình duyệt web. Nếu bạn đang sử dụng một trong các thư viện ứng dụng API, hãy xem các yêu cầu dành riêng cho ngôn ngữ bên dưới.

PHP (#php)Python (#python)Ruby (#ruby)Node.js (#node.js)HTTP/REST
(#httprest)

Bạn không cần cài đặt bất kỳ thư viện nào để có thể gọi trực tiếp các điểm cuối OAuth 2.0.

Lấy mã truy cập OAuth 2.0

Các bước sau đây cho biết cách ứng dụng của bạn tương tác với máy chủ OAuth 2.0 của Google để nhận được sự đồng ý của người dùng nhằm thực hiện một yêu cầu API thay cho người dùng. Ứng dụng của bạn phải có sự đồng ý đó thì mới có thể thực thi yêu cầu Google API cần có sự uỷ quyền của người dùng.

Danh sách dưới đây tóm tắt nhanh các bước này:

1. Ứng dụng của bạn xác định những quyền mà ứng dụng cần.
2. Ứng dụng của bạn sẽ chuyển hướng người dùng đến Google cùng với danh sách các quyền được yêu cầu.
3. Người dùng quyết định có cấp quyền cho ứng dụng của bạn hay không.
4. Ứng dụng của bạn sẽ biết được quyết định của người dùng.
5. Nếu người dùng cấp các quyền được yêu cầu, ứng dụng của bạn sẽ truy xuất các mã thông báo cần thiết để thực hiện các yêu cầu API thay cho người dùng.

Bước 1: Đặt các tham số uỷ quyền

Bước đầu tiên là tạo yêu cầu uỷ quyền. Yêu cầu đó đặt các thông số xác định ứng dụng của bạn và xác định các quyền mà người dùng sẽ được yêu cầu cấp cho ứng dụng của bạn.

- Nếu sử dụng một thư viện ứng dụng Google để xác thực và uỷ quyền OAuth 2.0, bạn sẽ tạo và định cấu hình một đối tượng xác định các tham số này.

- Nếu gọi trực tiếp điểm cuối Google OAuth 2.0, bạn sẽ tạo một URL và đặt các tham số trên URL đó.

Các thẻ bên dưới xác định các thông số uỷ quyền được hỗ trợ cho các ứng dụng máy chủ web. Các ví dụ dành riêng cho ngôn ngữ cũng cho thấy cách sử dụng thư viện ứng dụng hoặc thư viện uỷ quyền để định cấu hình một đối tượng đặt các tham số đó.

PHP (#php)Python (#python)Ruby (#ruby)Node.js (#node.js)HTTP/REST (#httprest)

Điểm cuối OAuth 2.0 của Google là

https://accounts.google.com/o/oauth2/v2/auth. Chỉ có thể truy cập vào điểm cuối này qua HTTPS. Các kết nối HTTP thông thường sẽ bị từ chối.

Máy chủ uỷ quyền của Google hỗ trợ các tham số chuỗi truy vấn sau đây cho các ứng dụng máy chủ web:

Thông số

client_id	Bắt buộc Mã ứng dụng khách cho ứng dụng của bạn. Bạn có thể tìm thấy giá trị này Cloud Console Clients page (https://console.developers.google.com/auth/clients?hl=vi).
redirect_uri	Bắt buộc Xác định vị trí mà máy chủ API chuyển hướng người dùng sau khi người dùng hoàn tất quy trình uỷ quyền. Giá trị này phải khớp chính xác với một trong c URI chuyển hướng được uỷ quyền cho ứng dụng OAuth 2.0 mà bạn đã định hình trong Cloud Console Clients page (https://console.developers.google.com/auth/clients?hl=vi) của ứng dụng giá trị này không khớp với một URI chuyển hướng được uỷ quyền cho client_id đã cung cấp, bạn sẽ gặp lỗi redirect_uri_mismatch. Xin lưu ý rằng lược đồ, trường hợp và dấu gạch chéo ở cuối ("/") của http https đều phải khớp.
response_type	Bắt buộc Xác định xem điểm cuối Google OAuth 2.0 có trả về mã uỷ quyền hay không. Đặt giá trị tham số thành code cho các ứng dụng máy chủ web.

Thông số

scope

Bắt buộc

Một danh sách các phạm vi được phân tách bằng dấu cách, xác định những nguyên nhân mà ứng dụng của bạn có thể truy cập thay cho người dùng. Những trị này thông báo cho màn hình đồng ý mà Google hiển thị cho người dùng.

Phạm vi cho phép ứng dụng của bạn chỉ yêu cầu quyền truy cập vào những nguyên nhân cần thiết, đồng thời cho phép người dùng kiểm soát mức độ truy cập mà họ cấp cho ứng dụng của bạn. Do đó, có mối quan hệ tỷ lệ nghịch giữa lượng phạm vi được yêu cầu và khả năng nhận được sự đồng ý của người dùng.

Bạn nên yêu cầu ứng dụng truy cập vào các phạm vi uỷ quyền trong ngữ cảnh bất cứ khi nào có thể. Bằng cách yêu cầu quyền truy cập vào dữ liệu người dùng trong bối cảnh, sử dụng uỷ quyền từng phần (`#incrementalAuth`), bạn người dùng hiểu được lý do ứng dụng của bạn cần quyền truy cập mà ứng dụng đang yêu cầu.

access_type

Recommended (Nên dùng)

Cho biết liệu ứng dụng của bạn có thể làm mới mã truy cập khi người dùng không có mặt trên trình duyệt hay không. Các giá trị tham số hợp lệ là **online** (giá trị mặc định) và **offline**.

Đặt giá trị thành **offline** nếu ứng dụng của bạn cần làm mới mã truy cập người dùng không có mặt trên trình duyệt. Đây là phương thức làm mới mã thông báo truy cập được mô tả ở phần sau của tài liệu này. Giá trị này hướng dẫn máy chủ uỷ quyền của Google trả về mã làm mới và mã truy cập vào lần đầu tiên ứng dụng của bạn trao đổi mã uỷ quyền để lấy mã thông báo.

state

Recommended (Nên dùng)

Chỉ định mọi giá trị chuỗi mà ứng dụng của bạn dùng để duy trì trạng thái yêu cầu uỷ quyền và phản hồi của máy chủ uỷ quyền. Máy chủ trả về chính giá trị mà bạn gửi dưới dạng cặp **name=value** trong thành phần truy vấn cuối (**?**) của **redirect_uri** sau khi người dùng đồng ý hoặc từ chối yêu cầu truy cập của ứng dụng.

Bạn có thể sử dụng tham số này cho nhiều mục đích, chẳng hạn như chuyển hướng người dùng đến tài nguyên chính xác trong ứng dụng của bạn, gửi sử dụng một lần và giảm thiểu hành vi giả mạo yêu cầu trên nhiều trang web. **redirect_uri** của bạn có thể bị đoán, nên việc sử dụng giá trị **state** có giúp bạn yên tâm hơn rằng một kết nối đến là kết quả của một yêu cầu xác thực. Nếu tạo một chuỗi ngẫu nhiên hoặc mã hoá hàm băm của cookie hoặc một giá trị khác ghi lại trạng thái của ứng dụng, bạn có thể xác thực phản hồi.

Thông số

để đảm bảo thêm rằng yêu cầu và phản hồi bắt nguồn từ cùng một trình di cung cấp biện pháp bảo vệ chống lại các cuộc tấn công như giả mạo yêu cầu trên nhiều trang web

(<https://datatracker.ietf.org/doc/html/rfc6749#section-10.12>). Hãy xem tài liệu về OpenID Connect

(<https://developers.google.com/identity/protocols/oauth2/openid-connect#hl=vi#createxsrtoken>)

để biết ví dụ về cách tạo và xác nhận mã thông báo **state**.



Lưu ý quan trọng: Ứng dụng OAuth phải ngăn chặn CSRF như được nêu trong Quy cách OAuth2

(<https://datatracker.ietf.org/doc/html/rfc6749#section-10.12>). Một cách để đạt được điều này là sử dụng tham số **state** để duy trì trạng thái giữa yêu cầu ủy quyền và phản hồi của máy chủ ủy quyền.

include_granted_scopes

Không bắt buộc

Cho phép các ứng dụng sử dụng tính năng ủy quyền từng phần để yêu cầu quyền truy cập vào các phạm vi bổ sung trong ngữ cảnh. Nếu bạn đặt giá trị của tham số này thành **true** và yêu cầu ủy quyền được cấp, thì mã truy cập mới cũng sẽ bao gồm mọi phạm vi mà người dùng đã cấp quyền truy cập cho ứng dụng trước đó. Hãy xem phần ủy quyền gia tăng (#incrementalAuth) để biết ví dụ.

enable_granular_consent

Không bắt buộc

Giá trị mặc định là **true**. Nếu được đặt thành **false**, các quyền chi tiết học với Tài khoản Google

(https://developers.google.com/identity/protocols/oauth2/resources/granular_permissions?hl=vi)

sẽ bị vô hiệu hoá đối với những mã ứng dụng OAuth được tạo trước năm 2018. Không ảnh hưởng đến mã ứng dụng OAuth mới hơn, vì các quyền chi tiết học luôn được bật cho các mã này.

Khi Google bật quyền chi tiết cho một ứng dụng, tham số này sẽ không còn hiệu lực.

login_hint

Không bắt buộc

Nếu biết người dùng nào đang cố gắng xác thực, ứng dụng của bạn có thể sử dụng tham số này để cung cấp một gợi ý cho Máy chủ xác thực của Google. Máy chủ sử dụng gợi ý này để đơn giản hoá quy trình đăng nhập bằng cách

Thông số

điền sẵn trường email trong biểu mẫu đăng nhập hoặc bằng cách chọn ph
đăng nhập nhiều lần thích hợp.

Đặt giá trị tham số thành địa chỉ email hoặc giá trị nhận dạng **sub**, tương
đương với mã nhận dạng Google của người dùng.

prompt

Không bắt buộc

Một danh sách phân tách bằng dấu cách, phân biệt chữ hoa chữ thường g
các câu lệnh để trình bày cho người dùng. Nếu bạn không chỉ định tham số
người dùng sẽ chỉ được nhắc vào lần đầu tiên dự án của bạn yêu cầu quye
truy cập. Hãy xem phần [Yêu cầu người dùng đồng ý lại](#)

(<https://developers.google.com/identity/protocols/oauth2/openid-connect#hl=vi#re-consent>)

để biết thêm thông tin.

Các giá trị có thể là:

none	Không hiển thị bất kỳ màn hình xác thực hoặc đồng ý nào. Không được chỉ định c với các giá trị khác.
consent	Yêu cầu người dùng đồng ý.
select_account	Nhắc người dùng chọn một tài khoản.

Bước 2: Chuyển hướng đến máy chủ OAuth 2.0 của Google

Chuyển hướng người dùng đến máy chủ OAuth 2.0 của Google để bắt đầu quy trình xác thực và
ủy quyền. Thông thường, điều này xảy ra khi ứng dụng của bạn cần truy cập vào dữ liệu của
người dùng lần đầu tiên. Trong trường hợp [ủy quyền gia tăng](#) (#incrementalAuth), bước này cũng
xảy ra khi ứng dụng của bạn cần truy cập vào các tài nguyên bổ sung mà ứng dụng chưa có
quyền truy cập.

[PHP](#) (#php)[Python](#) (#python)[Ruby](#) (#ruby)[Node.js](#) (#node.js)[HTTP/REST](#)
(#httprest)

Ví dụ về việc chuyển hướng đến máy chủ ủy quyền của Google

Sau đây là một ví dụ về URL, có dấu ngắt dòng và khoảng trắng để dễ đọc.

```
https://accounts.google.com/o/oauth2/v2/auth?
scope=https%3A//www.googleapis.com/auth/drive.metadata.readonly%20https%3A//
access_type=offline&
include_granted_scopes=true&
response_type=code&
state=state_parameter_passthrough_value&
redirect_uri=https%3A//developers.google.com/oauthplayground&
client_id=client_id
```

Sau khi bạn tạo URL yêu cầu, hãy chuyển hướng người dùng đến URL đó.

Máy chủ OAuth 2.0 của Google xác thực người dùng và xin sự đồng ý của người dùng để ứng dụng của bạn truy cập vào các phạm vi được yêu cầu. Phản hồi được gửi lại cho ứng dụng của bạn bằng URL chuyển hướng mà bạn đã chỉ định.

Bước 3: Google yêu cầu người dùng đồng ý

Trong bước này, người dùng sẽ quyết định có cấp cho ứng dụng của bạn quyền truy cập được yêu cầu hay không. Ở giai đoạn này, Google sẽ hiển thị một cửa sổ đồng ý cho biết tên của ứng dụng và các dịch vụ Google API mà ứng dụng đang yêu cầu quyền truy cập bằng thông tin xác thực uỷ quyền của người dùng, cũng như bản tóm tắt về các phạm vi truy cập sẽ được cấp. Sau đó, người dùng có thể đồng ý cấp quyền truy cập vào một hoặc nhiều phạm vi mà ứng dụng của bạn yêu cầu hoặc từ chối yêu cầu đó.

Ứng dụng của bạn không cần làm gì ở giai đoạn này vì ứng dụng sẽ đợi phản hồi từ máy chủ OAuth 2.0 của Google cho biết có quyền truy cập nào được cấp hay không. Phản hồi đó được giải thích trong bước tiếp theo.

Lỗi

Các yêu cầu đến điểm cuối uỷ quyền OAuth 2.0 của Google có thể hiển thị thông báo lỗi cho người dùng thay vì các luồng xác thực và uỷ quyền dự kiến. Sau đây là các mã lỗi thường gặp và cách khắc phục được đề xuất:

`admin_policy_enforced`

Tài khoản Google không thể ủy quyền cho một hoặc nhiều phạm vi được yêu cầu do các chính sách của quản trị viên Google Workspace. Hãy xem bài viết trợ giúp dành cho Quản trị viên Google Workspace [Kiểm soát việc những ứng dụng nội bộ và ứng dụng của bên thứ ba nào truy cập vào dữ liệu Google Workspace](https://support.google.com/a/answer/7281227?hl=vi) (<https://support.google.com/a/answer/7281227?hl=vi>) để biết thêm thông tin về cách quản trị viên có thể hạn chế quyền truy cập vào tất cả các phạm vi hoặc phạm vi nhạy cảm và bị hạn chế cho đến khi quyền truy cập được cấp rõ ràng cho mã ứng dụng OAuth của bạn.

`disallowed_useragent`

Điểm cuối ủy quyền xuất hiện trong một tác nhân người dùng được nhúng mà [Chính sách về OAuth 2.0](https://developers.google.com/identity/protocols/oauth2/policies?hl=vi#browsers) (<https://developers.google.com/identity/protocols/oauth2/policies?hl=vi#browsers>) của Google không cho phép.

Nhà phát triển iOS và macOS có thể gặp phải lỗi này khi mở yêu cầu ủy quyền trong [WKWebView](https://developer.apple.com/documentation/webkit/wkwebview) (<https://developer.apple.com/documentation/webkit/wkwebview>). Thay vào đó, nhà phát triển nên dùng các thư viện iOS như [Google Sign-In for iOS](https://developers.google.com/identity/sign-in/ios?hl=vi) (<https://developers.google.com/identity/sign-in/ios?hl=vi>) hoặc [AppAuth for iOS](https://openid.github.io/AppAuth-iOS/) (<https://openid.github.io/AppAuth-iOS/>) của OpenID Foundation.

Nhà phát triển web có thể gặp phải lỗi này khi một ứng dụng iOS hoặc macOS mở một đường liên kết chung trên web trong một tác nhân người dùng được nhúng và người dùng chuyển đến điểm cuối ủy quyền OAuth 2.0 của Google từ trang web của bạn. Nhà phát triển nên cho phép các đường liên kết chung mở trong trình xử lý đường liên kết mặc định của hệ điều hành, bao gồm cả trình xử lý [Đường liên kết phổ quát](https://developer.apple.com/ios/universal-links/) (<https://developer.apple.com/ios/universal-links/>) hoặc ứng dụng trình duyệt mặc định. Thư viện [SFSafariViewController](https://developer.apple.com/documentation/safariservices/sfsafariviewcontroller) (<https://developer.apple.com/documentation/safariservices/sfsafariviewcontroller>) cũng là một lựa chọn được hỗ trợ.

`org_internal`

Mã ứng dụng OAuth trong yêu cầu thuộc một dự án giới hạn quyền truy cập vào Tài khoản Google trong một [Tổ chức Google Cloud](https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy?hl=vi#organizations) (<https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy?hl=vi#organizations>)

cụ thể. Để biết thêm thông tin về lựa chọn cấu hình này, hãy xem phần [Loại người dùng](https://support.google.com/cloud/answer/10311615?hl=vi#user-type) (<https://support.google.com/cloud/answer/10311615?hl=vi#user-type>) trong bài viết trợ giúp Thiết lập màn hình đồng ý của OAuth.

invalid_client

Mật khẩu ứng dụng OAuth không chính xác. Xem xét [cấu hình ứng dụng OAuth](https://developers.google.com/identity/protocols/oauth2/javascript-implicit-flow?hl=vi#creatingcred)

(<https://developers.google.com/identity/protocols/oauth2/javascript-implicit-flow?hl=vi#creatingcred>), bao gồm cả mã ứng dụng và khoá bí mật được dùng cho yêu cầu này.

deleted_client

Ứng dụng OAuth đang được dùng để đưa ra yêu cầu đã bị xoá. Bạn có thể xoá theo cách thủ công hoặc tự động trong trường hợp [các ứng dụng không dùng đến](https://support.google.com/cloud/answer/15549257?hl=vi#unused-client-deletion)

(<https://support.google.com/cloud/answer/15549257?hl=vi#unused-client-deletion>). Bạn có thể khôi phục các ứng dụng khách đã xoá trong vòng 30 ngày kể từ ngày xoá. [Tìm hiểu thêm](https://support.google.com/cloud/answer/15549257?hl=vi#delete-oauth-clients) (<https://support.google.com/cloud/answer/15549257?hl=vi#delete-oauth-clients>).

invalid_grant

Khi làm mới mã truy cập (#offline) hoặc sử dụng [ủy quyền gia tăng](#) (#incrementalAuth), mã thông báo có thể đã hết hạn hoặc đã bị vô hiệu hoá. Xác thực lại người dùng và yêu cầu người dùng đồng ý để lấy mã thông báo mới. Nếu bạn vẫn gặp lỗi này, hãy đảm bảo rằng ứng dụng của bạn đã được định cấu hình đúng cách và bạn đang sử dụng đúng mã thông báo và tham số trong yêu cầu. Nếu không, tài khoản người dùng có thể đã bị xoá hoặc vô hiệu hoá.

redirect_uri_mismatch

`redirect_uri` được truyền trong yêu cầu ủy quyền không khớp với URI chuyển hướng được ủy quyền cho mã ứng dụng OAuth. Xem xét các URI chuyển hướng được ủy quyền trong Google Cloud Console [Clients page](https://console.developers.google.com/auth/clients?hl=vi) (<https://console.developers.google.com/auth/clients?hl=vi>).

Tham số `redirect_uri` có thể đề cập đến quy trình OAuth ngoài băng tần (OOB) đã ngừng hoạt động và không còn được hỗ trợ nữa. Tham khảo [hướng dẫn di chuyển](https://developers.google.com/identity/protocols/oauth2/resources/oob-migration?hl=vi) (<https://developers.google.com/identity/protocols/oauth2/resources/oob-migration?hl=vi>) để cập nhật quy trình tích hợp.

invalid_request

Đã xảy ra lỗi với yêu cầu bạn gửi. Điều này có thể là do một số nguyên nhân:

- Yêu cầu không được định dạng đúng cách

- Yêu cầu thiếu các tham số bắt buộc
- Yêu cầu sử dụng một phương thức uỷ quyền mà Google không hỗ trợ. Xác minh rằng quá trình tích hợp OAuth của bạn sử dụng một phương thức tích hợp được đề xuất

Bước 4: Xử lý phản hồi của máy chủ OAuth 2.0

Lưu ý quan trọng: Trước khi xử lý phản hồi OAuth 2.0 trên máy chủ, bạn nên xác nhận rằng **state** nhận được từ Google khớp với **state** được gửi trong yêu cầu uỷ quyền. Quy trình xác minh này giúp đảm bảo rằng người dùng, chứ không phải một tập lệnh độc hại, đang đưa ra yêu cầu và giảm nguy cơ tấn công CSRF (<https://datatracker.ietf.org/doc/html/rfc6749#section-10.12>).

Máy chủ OAuth 2.0 sẽ phản hồi yêu cầu truy cập của ứng dụng bằng cách sử dụng URL được chỉ định trong yêu cầu.

Nếu người dùng phê duyệt yêu cầu truy cập, thì phản hồi sẽ chứa mã uỷ quyền. Nếu người dùng không phê duyệt yêu cầu, thì phản hồi sẽ chứa thông báo lỗi. Mã uỷ quyền hoặc thông báo lỗi được trả về cho máy chủ web sẽ xuất hiện trên chuỗi truy vấn, như minh họa dưới đây:

Phản hồi lỗi:

```
https://oauth2.example.com/auth?error=access_denied
```

Phản hồi mã uỷ quyền:

```
https://oauth2.example.com/auth?code=4/P7q7W91a-oMsCeLvIaQm6bTrgtp7
```

Lưu ý quan trọng: Nếu điểm cuối phản hồi của bạn hiển thị một trang HTML, thì mọi tài nguyên trên trang đó đều có thể thấy mã uỷ quyền trong URL. Các tập lệnh có thể đọc trực tiếp URL và URL trong tiêu đề HTTP **Referer** có thể được gửi đến bất kỳ hoặc tất cả tài nguyên trên trang.

Hãy cân nhắc kỹ xem bạn có muốn gửi thông tin uỷ quyền đến tất cả tài nguyên trên trang đó hay không (đặc biệt là các tập lệnh của bên thứ ba, chẳng hạn như trình hỗ trợ xã hội và số liệu phân tích). Để tránh vấn đề

này, bạn nên yêu cầu máy chủ xử lý yêu cầu trước, sau đó chuyển hướng đến một URL khác không có các tham số phản hồi.

Phản hồi mẫu của máy chủ OAuth 2.0

Bạn có thể kiểm thử quy trình này bằng cách nhấp vào URL mẫu sau. URL này yêu cầu quyền truy cập chỉ đọc để xem siêu dữ liệu của các tệp trong Google Drive và quyền truy cập chỉ đọc để xem các sự kiện trên Lịch Google:

https://accounts.google.com/o/oauth2/v2/auth?scope=https%3A//www.googleapis.com/auth/drive.metadata.readonly%20https%3A//www.access_type=offline&include_granted_scopes=true&response_type=code&state=state_parameter_passthrough_value&redirect_uri=https%3A//developers.google.com/oauthplayground&client_id=client_id (https://accounts.google.com/o/oauth2/v2/auth?scope=https%3A%2F%2Fwww.!

Sau khi hoàn tất quy trình OAuth 2.0, trình duyệt sẽ chuyển hướng bạn đến [OAuth 2.0 Playground](https://developers.google.com/oauthplayground/?hl=vi) (https://developers.google.com/oauthplayground/?hl=vi), một công cụ để kiểm thử các quy trình OAuth. Bạn sẽ thấy OAuth 2.0 Playground đã tự động ghi lại mã ủy quyền.

Bước 5: Trao đổi mã ủy quyền để lấy mã làm mới và mã truy cập

Sau khi nhận được mã ủy quyền, máy chủ web có thể trao đổi mã ủy quyền để lấy mã truy cập.

PHP (#php) Python (#python) Ruby (#ruby) Node.js (#node.js) HTTP/REST (#httprest)	
Để đổi mã ủy quyền lấy mã truy cập, hãy gọi điểm cuối https://oauth2.googleapis.com/token và đặt các tham số sau:	
Trường	
client_id	Mã ứng dụng khách nhận được từ Cloud Console Clients page (https://console.developers.google.com/auth/clients?hl=vi).

Trường	
client_secret	Không bắt buộc Khoá bí mật của ứng dụng khách nhận được từ Cloud Console Clients page (https://console.developers.google.com/auth/clients?hl=vi).
code	Mã uỷ quyền được trả về từ yêu cầu ban đầu.
grant_type	<u>Như được xác định trong quy cách OAuth 2.0</u> (https://tools.ietf.org/html/rfc6749#section-4.1.3), bạn phải đặt giá trị của trường này thành authorization_code.
redirect_uri	Một trong các URI chuyển hướng được liệt kê cho dự án của bạn trong Cloud Console Clients page (https://console.developers.google.com/auth/clients?hl=vi) cho client_id đã cho.
Đoạn mã sau đây cho thấy một yêu cầu mẫu: <pre>POST /token HTTP/1.1 Host: oauth2.googleapis.com Content-Type: application/x-www-form-urlencoded code=4/P7q7W91a-oMsCeLvIaQm6bTrgtp7& client_id=<i>your_client_id</i>& redirect_uri=https%3A//developers.google.com/oauthplayground& grant_type=authorization_code</pre> Google phản hồi yêu cầu này bằng cách trả về một đối tượng JSON chứa mã truy cập có thời hạn ngắn và mã làm mới. Xin lưu ý rằng mã làm mới chỉ được trả về nếu ứng dụng của bạn đặt tham số access_type thành offline trong <u>yêu cầu ban đầu đến máy chủ uỷ quyền của Google</u> (#creatingclient). Phản hồi chứa các trường sau:	
Trường	
access_token	Mã thông báo mà ứng dụng của bạn gửi để uỷ quyền cho một yêu cầu API của Google.

Trường	
expires_in	Thời gian còn lại của mã truy cập tính bằng giây.
refresh_token	Mã thông báo mà bạn có thể dùng để lấy mã thông báo truy cập mới. Mã làm mới có hiệu lực cho đến khi người dùng thu hồi quyền truy cập hoặc mã làm mới hết hạn. Xin lưu ý rằng trường này chỉ xuất hiện trong phản hồi này nếu bạn đặt tham số access_type thành offline trong yêu cầu ban đầu gửi đến máy chủ uỷ quyền của Google.
refresh_token_expires_in	Thời gian tồn tại còn lại của mã làm mới tính bằng giây. Giá trị này chỉ được đặt khi người dùng cấp <u>quyền truy cập dựa trên thời gian</u> (https://developers.google.com/identity/protocols/oauth2/web-server?hl=vi#time-based-access)
scope	Phạm vi truy cập do access_token cấp dưới dạng danh sách các chuỗi phân biệt chữ hoa/chữ thường, được phân tách bằng dấu cách.
token_type	Loại mã thông báo được trả về. Tại thời điểm này, giá trị của trường này luôn được đặt thành Bearer .

Quan trọng: Ứng dụng của bạn phải lưu trữ cả hai mã thông báo ở một vị trí an toàn, lâu dài mà ứng dụng có thể truy cập được giữa các lệnh gọi khác nhau. Mã thông báo làm mới cho phép ứng dụng của bạn lấy mã thông báo truy cập mới nếu mã thông báo hiện tại của bạn hết hạn. Do đó, nếu ứng dụng của bạn mất mã làm mới, người dùng sẽ cần lặp lại quy trình đồng ý OAuth 2.0 để ứng dụng của bạn có thể lấy mã làm mới mới.

Đoạn mã sau đây cho thấy một phản hồi mẫu:

```
{
  "access_token": "1/ffAGRNJru1FTz70BzhT3Zg",
  "expires_in": 3920,
  "token_type": "Bearer",
  "scope": "https://www.googleapis.com/auth/drive.metadata.readonly https:",
  "refresh_token": "1//xEoDL4iW3cx1I7yDbSRFYNG01kVKM2C-259H0F2aQbI"
}
```

◆ Hướng dẫn viết mã



➤ **Lưu ý:** Ứng dụng của bạn sẽ bỏ qua mọi trường không xác định có trong phản hồi.

Lỗi

Khi trao đổi mã ủy quyền để lấy mã truy cập, bạn có thể gặp phải lỗi sau thay vì phản hồi dự kiến. Dưới đây là danh sách mã lỗi thường gặp và các giải pháp được đề xuất.

`invalid_grant`

Mã ủy quyền bạn cung cấp không hợp lệ hoặc có định dạng không chính xác. Yêu cầu mã mới bằng cách [khởi động lại quy trình OAuth](#) (#creatingclient) để nhắc người dùng đồng ý lại.

Bước 6: Kiểm tra xem người dùng đã cấp quyền truy cập vào những phạm vi nào

Khi yêu cầu **nhieu** quyền (phạm vi), người dùng có thể không cấp cho ứng dụng của bạn quyền truy cập vào tất cả các quyền đó. Ứng dụng của bạn phải xác minh những phạm vi nào đã được cấp và xử lý một cách linh hoạt những trường hợp một số quyền bị từ chối, thường là bằng cách tắt các tính năng dựa vào những phạm vi bị từ chối đó.

Tuy nhiên, vẫn có một số trường hợp ngoại lệ. Các ứng dụng Google Workspace Enterprise có [quyền ủy quyền trên toàn miền](#) (<https://support.google.com/a/answer/162106?hl=vi>) hoặc các ứng dụng được đánh dấu là [Đáng tin cậy](#).

(<https://support.google.com/a/answer/7281227?hl=vi#zipppy=,change-access-from-the-app-list>) sẽ bỏ qua màn hình đồng ý cấp quyền chi tiết. Đối với những ứng dụng này, người dùng sẽ không thấy màn hình đồng ý cấp quyền chi tiết. Thay vào đó, ứng dụng của bạn sẽ nhận được tất cả các phạm vi được yêu cầu hoặc không nhận được phạm vi nào.

Để biết thêm thông tin chi tiết, hãy xem bài viết [Cách xử lý các quyền ở cấp độ chi tiết](#) (<https://developers.google.com/identity/protocols/oauth2/resources/granular-permissions?hl=vi>).

[PHP](#) (#php)[Python](#) (#python)[Ruby](#) (#ruby)[Node.js](#) (#node.js)[HTTP/REST](#) (#httprest)

Để kiểm tra xem người dùng đã cấp cho ứng dụng của bạn quyền truy cập vào một phạm vi cụ thể hay chưa, hãy kiểm tra trường `scope` trong phản hồi mã truy cập. Phạm vi

truy cập do access_token cấp, được biểu thị dưới dạng danh sách các chuỗi phân biệt chữ hoa chữ thường và phân tách bằng dấu cách.

Ví dụ: phản hồi mã thông báo truy cập mẫu sau đây cho biết rằng người dùng đã cấp cho ứng dụng của bạn quyền truy cập vào quyền chỉ đọc đối với hoạt động trên Drive và sự kiện trên Lịch:

```
{
  "access_token": "1/ffAGRNJru1FTz70BzhT3Zg",
  "expires_in": 3920,
  "token_type": "Bearer",
  "scope": "https://www.googleapis.com/auth/drive.metadata.readonly https://www.googleapis.com/auth/calendar",
  "refresh_token": "1//xEoDL4iW3cx1I7yDbSRFYNG01kVKM2C-259H0F2aQbI"
}
```

Gọi API của Google

[PHP](#) (#php) [Python](#) (#python) [Ruby](#) (#ruby) [Node.js](#) (#node.js) [HTTP/REST](#) (#httprest)

Sau khi ứng dụng của bạn nhận được mã truy cập, bạn có thể dùng mã này để thực hiện các lệnh gọi đến một API của Google thay cho một tài khoản người dùng nhất định nếu(các) phạm vi truy cập mà API yêu cầu đã được cấp. Để thực hiện việc này, hãy thêm mã truy cập vào yêu cầu gửi đến API bằng cách thêm tham số truy vấn `access_token` hoặc giá trị tiêu đề HTTP `Authorization Bearer`. Khi có thể, bạn nên dùng tiêu đề HTTP vì chuỗi truy vấn thường xuất hiện trong nhật ký máy chủ. Trong hầu hết các trường hợp, bạn có thể sử dụng một thư viện ứng dụng để thiết lập các lệnh gọi đến API của Google (ví dụ: khi gọi [API Tập trên Drive](#) (<https://developers.google.com/drive/api/v2/reference?hl=vi#Files>)).

Bạn có thể dùng thử tất cả các API của Google và xem phạm vi của các API đó tại [OAuth 2.0 Playground](#) (<https://developers.google.com/oauthplayground/?hl=vi>).

Ví dụ về HTTP GET

Một lệnh gọi đến điểm cuối **drive.files**

(<https://developers.google.com/drive/v2/reference/files/list?hl=vi>) (API Tập trên Drive) bằng cách sử dụng tiêu đề HTTP **Authorization: Bearer** có thể trông như sau. Xin lưu ý rằng bạn cần chỉ định mã truy cập của riêng mình:

```
GET /drive/v2/files HTTP/1.1
Host: www.googleapis.com
Authorization: Bearer access_token
```

Sau đây là một lệnh gọi đến cùng một API cho người dùng đã xác thực bằng cách sử dụng tham số chuỗi truy vấn `access_token`:

```
GET https://www.googleapis.com/drive/v2/files?access_token=access_token
```

Ví dụ về curl

Bạn có thể kiểm thử các lệnh này bằng ứng dụng dòng lệnh `curl`. Sau đây là ví dụ sử dụng lựa chọn tiêu đề HTTP (nên dùng):

```
$ curl -H "Authorization: Bearer access_token" https://www.googleapis.com
```

Hoặc, bạn có thể chọn tham số chuỗi truy vấn:

```
$ curl https://www.googleapis.com/drive/v2/files?access_token=access_token
```

Ví dụ đầy đủ

Ví dụ sau đây in danh sách tệp có định dạng JSON trong Google Drive của người dùng sau khi người dùng xác thực và đồng ý cho ứng dụng truy cập vào siêu dữ liệu Drive của người dùng.

[PHP](#) (#php)[Python](#) (#python)[Ruby](#) (#ruby)[Node.js](#) (#node.js)[HTTP/REST](#) (#httprest)

Ví dụ này bằng Python sử dụng khung [Flask](https://palletsprojects.com/p/flask/) (<https://palletsprojects.com/p/flask/>) và thư viện [Requests](https://requests.readthedocs.io/) (<https://requests.readthedocs.io/>) để minh họa quy trình web OAuth 2.0. Bạn nên dùng Thư viện ứng dụng API của Google cho Python cho quy trình này. (Ví dụ trong thẻ Python có sử dụng thư viện ứng dụng.)

```
import json
import flask
import requests

app = flask.Flask(__name__)

# To get these credentials (CLIENT_ID CLIENT_SECRET) and for your application:
# https://console.cloud.google.com/apis/credentials.
CLIENT_ID = '123456789.apps.googleusercontent.com'
CLIENT_SECRET = 'abc123' # Read from a file or environmental variable in a future version.

# Access scopes for two non-Sign-In scopes: Read-only Drive activity and Google Drive API
SCOPE = 'https://www.googleapis.com/auth/drive.metadata.readonly https://www.googleapis.com/auth/drive'

# Indicate where the API server will redirect the user after the user completes
# the authorization flow. The redirect URI is required. The value must exactly
# match one of the authorized redirect URIs for the OAuth 2.0 client, which is
# configured in the API Console. If this value doesn't match an authorized
# you will get a 'redirect_uri_mismatch' error.
REDIRECT_URI = 'http://example.com/oauth2callback'

@app.route('/')
def index():
    if 'credentials' not in flask.session:
        return flask.redirect(flask.url_for('oauth2callback'))

    credentials = json.loads(flask.session['credentials'])

    if credentials['expires_in'] <= 0:
        return flask.redirect(flask.url_for('oauth2callback'))
    else:
        # User authorized the request. Now, check which scopes were granted.
        if 'https://www.googleapis.com/auth/drive.metadata.readonly' in credentials['scope']:
            # User authorized read-only Drive activity permission.
            # Example of using Google Drive API to list filenames in user's Drive
```

```

        headers = {'Authorization': 'Bearer {}'.format(credentials['access_token'])}
        req_uri = 'https://www.googleapis.com/drive/v2/files'
        r = requests.get(req_uri, headers=headers).text
    else:
        # User didn't authorize read-only Drive activity permission.
        # Update UX and application accordingly
        r = 'User did not authorize Drive permission.'

    # Check if user authorized Calendar read permission.
    if 'https://www.googleapis.com/auth/calendar.readonly' in credentials:
        # User authorized Calendar read permission.
        # Calling the APIs, etc.
        r += 'User authorized Calendar permission.'
    else:
        # User didn't authorize Calendar read permission.
        # Update UX and application accordingly
        r += 'User did not authorize Calendar permission.'

    return r

@app.route('/oauth2callback')
def oauth2callback():
    if 'code' not in flask.request.args:
        state = str(uuid.uuid4())
        flask.session['state'] = state
        # Generate a url that asks permissions for the Drive activity
        # and Google Calendar scope. Then, redirect user to the url.
        auth_uri = ('https://accounts.google.com/o/oauth2/v2/auth?response_type=code&client_id={}&redirect_uri={}&scope={}&state={}'.format(CLIENT_ID, REDIRECT_URI,
                                                                                                     'drive https://www.googleapis.com/auth/calendar.readonly')).format(CLIENT_ID, REDIRECT_URI,
                                                                                                     'drive https://www.googleapis.com/auth/calendar.readonly')

        return flask.redirect(auth_uri)
    else:
        if 'state' not in flask.request.args or flask.request.args['state'] != flask.session['state']:
            return 'State mismatch. Possible CSRF attack.', 400

        auth_code = flask.request.args.get('code')
        data = {'code': auth_code,
                'client_id': CLIENT_ID,
                'client_secret': CLIENT_SECRET,
                'redirect_uri': REDIRECT_URI,
                'grant_type': 'authorization_code'}

        # Exchange authorization code for access and refresh tokens (if access.
        r = requests.post('https://oauth2.googleapis.com/token', data=data)
        flask.session['credentials'] = r.json()

```

```

        return flask.redirect(flask.url_for('index'))

if __name__ == '__main__':
    import uuid
    app.secret_key = str(uuid.uuid4())
    app.debug = False
    app.run()

```

◆ Hướng dẫn viết mã



Quy tắc xác thực URI chuyển hướng

Google áp dụng các quy tắc xác thực sau đây cho URI chuyển hướng để giúp nhà phát triển bảo mật ứng dụng của họ. URI chuyển hướng của bạn phải tuân thủ các quy tắc sau. Xem [mục 3 của RFC 3986](https://tools.ietf.org/html/rfc3986#section-3) (<https://tools.ietf.org/html/rfc3986#section-3>) để biết định nghĩa về miền, máy chủ lưu trữ, đường dẫn, truy vấn, lược đồ và thông tin người dùng được đề cập bên dưới.

Các quy tắc xác thực

<u>Lược đồ</u> (https://tools.ietf.org/html/rfc3986#section-3.1)	URI chuyển hướng phải sử dụng lược đồ HTTPS, không được sử dụng HTTP thuần túy. URI máy chủ cục bộ (bao gồm cả URI địa chỉ IP máy chủ cục bộ) được miễn áp dụng quy tắc này.
<u>Máy chủ lưu trữ</u> (https://tools.ietf.org/html/rfc3986#section-3.2.2)	Máy chủ lưu trữ không được là địa chỉ IP thô. Địa chỉ IP máy chủ cục bộ được miễn tuân thủ quy tắc này.
<u>Miền</u> (https://tools.ietf.org/html/rfc1034)	- TLD của máy chủ lưu trữ (<u>Miền cấp cao nhất</u> (https://tools.ietf.org/id/draft-liman-tld-names-00.html)) phải thuộc <u>danh sách hậu tố công khai</u> (https://publicsuffix.org/list/). - Miền lưu trữ không được là "googleusercontent.com". - URI chuyển hướng không được chứa miền rút gọn URL (ví dụ: goo.gl) trừ phi ứng dụng sở hữu miền đó. Ngoài ra, nếu một ứng dụng sở hữu miền rút gọn chọn chuyển hướng đến miền đó, thì URI chuyển hướng đó phải chứa "/google-

Các quy tắc xác thực

	<code>callback/</code> trong đường dẫn hoặc kết thúc bằng <code>/google-callback</code> .
<u>Userinfo</u> (https://tools.ietf.org/html/rfc3986#section-3.2.1)	URI chuyển hướng không được chứa thành phần phụ userinfo.
<u>Đường dẫn</u> (https://tools.ietf.org/html/rfc3986#section-3.3)	URI chuyển hướng không được chứa đường dẫn truyền tải (còn gọi là quay lại thư mục), được biểu thị bằng <code>/..</code> hoặc <code>\..</code> hoặc mã hoá URL của chúng.
<u>Cụm từ tìm kiếm</u> (https://tools.ietf.org/html/rfc3986#section-3.4)	URI chuyển hướng không được chứa <u>lệnh chuyển hướng mở</u> (https://tools.ietf.org/html/rfc6749#section-10.15).
<u>Mảnh</u> (https://tools.ietf.org/html/rfc3986#section-3.5)	URI chuyển hướng không được chứa thành phần đoạn.
Ký tự	URI chuyển hướng không được chứa một số ký tự, bao gồm: - Ký tự đại diện (<code>' * '</code>) - Ký tự ASCII không in được - Mã hoá phần trăm không hợp lệ (mọi mã hoá phần trăm không tuân theo dạng mã hoá URL của dấu phần trăm theo sau là hai chữ số thập lục phân) - Ký tự rỗng (một ký tự NULL được mã hoá, ví dụ: <code>%00</code>, <code>%C0%80</code>)

Ủy quyền theo tiến trình

Trong giao thức OAuth 2.0, ứng dụng của bạn yêu cầu cấp quyền truy cập vào các tài nguyên được xác định theo phạm vi. Bạn nên yêu cầu uỷ quyền cho các tài nguyên vào thời điểm cần thiết để mang lại trải nghiệm tốt nhất cho người dùng. Để cho phép hoạt động đó, máy chủ uỷ quyền của Google hỗ trợ việc uỷ quyền gia tăng. Tính năng này cho phép bạn yêu cầu các phạm vi khi cần và nếu người dùng cấp quyền cho phạm vi mới, thì tính năng này sẽ trả về một mã uỷ quyền có thể được trao đổi để lấy một mã thông báo chứa tất cả các phạm vi mà người dùng đã cấp cho dự án.

Ví dụ: một ứng dụng cho phép mọi người nghe thử các bản nhạc và tạo bản phối có thể chỉ cần rất ít tài nguyên tại thời điểm đăng nhập, có lẽ chỉ cần tên của người đăng nhập. Tuy nhiên, để lưu một bản phối hoàn chỉnh, bạn cần có quyền truy cập vào Google Drive của họ. Hầu hết mọi người sẽ thấy việc này là bình thường nếu họ chỉ được yêu cầu cấp quyền truy cập vào Google Drive tại thời điểm ứng dụng thực sự cần quyền truy cập đó.

Trong trường hợp này, tại thời điểm đăng nhập, ứng dụng có thể yêu cầu các phạm vi `openid` và `profile` để thực hiện quy trình đăng nhập cơ bản, sau đó yêu cầu phạm vi `https://www.googleapis.com/auth/drive.file` tại thời điểm yêu cầu đầu tiên để lưu một bản phối.

Để triển khai uỷ quyền gia tăng, bạn hoàn tất quy trình thông thường để yêu cầu mã truy cập nhưng phải đảm bảo rằng yêu cầu uỷ quyền bao gồm các phạm vi đã được cấp trước đó. Phương pháp này giúp ứng dụng của bạn không phải quản lý nhiều mã truy cập.

Các quy tắc sau đây áp dụng cho mã truy cập nhận được từ một quy trình uỷ quyền gia tăng:

- Bạn có thể dùng mã thông báo này để truy cập vào các tài nguyên tương ứng với bất kỳ phạm vi nào được đưa vào quy trình uỷ quyền kết hợp mới.
- Khi bạn dùng mã làm mới cho hoạt động uỷ quyền kết hợp để lấy mã truy cập, mã truy cập sẽ đại diện cho hoạt động uỷ quyền kết hợp và có thể dùng cho bất kỳ giá trị `scope` nào có trong phản hồi.
- Uỷ quyền kết hợp bao gồm tất cả các phạm vi mà người dùng đã cấp cho dự án API, ngay cả khi các quyền được yêu cầu từ các ứng dụng khác nhau. Ví dụ: nếu người dùng cấp quyền truy cập vào một phạm vi bằng cách sử dụng ứng dụng máy tính của một ứng dụng, sau đó cấp một phạm vi khác cho cùng một ứng dụng thông qua ứng dụng di động, thì uỷ quyền kết hợp sẽ bao gồm cả hai phạm vi.
- Nếu bạn thu hồi một mã thông báo đại diện cho một uỷ quyền kết hợp, thì quyền truy cập vào tất cả các phạm vi uỷ quyền đó thay mặt cho người dùng được liên kết sẽ bị thu hồi đồng thời.

Thận trọng: nếu chọn thêm các phạm vi được cấp, hệ thống sẽ tự động thêm các phạm vi mà người dùng đã cấp trước đó vào yêu cầu uỷ quyền của bạn. Một trang cảnh báo hoặc lỗi có thể xuất hiện nếu ứng dụng của bạn hiện chưa được phê duyệt để yêu cầu tất cả các phạm vi có thể được trả về trong phản hồi. Hãy xem phần [Ứng dụng chưa được xác minh](https://support.google.com/cloud/answer/7454865?hl=vi) (<https://support.google.com/cloud/answer/7454865?hl=vi>) để biết thêm thông tin.

các dịch vụ sao lưu hoặc thực hiện các hành động vào thời gian đã định trước cần có khả năng làm mới mã truy cập khi người dùng không có mặt. Kiểu truy cập mặc định có tên là **online**.

Các ứng dụng web phía máy chủ, ứng dụng đã cài đặt và thiết bị đều nhận được mã làm mới trong quá trình uỷ quyền. Mã làm mới thường không được dùng trong các ứng dụng web phía máy khách (JavaScript).

PHP (#php)**Python** (#python)**Ruby** (#ruby)**Node.js** (#node.js)**HTTP/REST** (#httprest)

Để làm mới mã truy cập, ứng dụng của bạn sẽ gửi một yêu cầu HTTPS POST đến máy chủ uỷ quyền của Google (<https://oauth2.googleapis.com/token>) có chứa các tham số sau:

Trường

client_id	Mã ứng dụng khách nhận được từ API Console (https://console.developers.google.com/?hl=vi).
client_secret	Không bắt buộc Khoá bí mật của ứng dụng khách nhận được từ API Console (https://console.developers.google.com/?hl=vi).
grant_type	Như được xác định trong quy cách OAuth 2.0 (https://tools.ietf.org/html/rfc6749#section-6), giá trị của trường này phải được đặt thành refresh_token .
refresh_token	Mã làm mới được trả về từ quá trình trao đổi mã uỷ quyền.

Đoạn mã sau đây cho thấy một yêu cầu mẫu:

```
POST /token HTTP/1.1
Host: oauth2.googleapis.com
Content-Type: application/x-www-form-urlencoded

client_id=your_client_id&
refresh_token=refresh_token&
grant_type=refresh_token
```

Miễn là người dùng chưa thu hồi quyền truy cập đã cấp cho ứng dụng, máy chủ mã thông báo sẽ trả về một đối tượng JSON chứa mã thông báo truy cập mới. Đoạn mã sau đây cho thấy một phản hồi mẫu:

```
{
  "access_token": "1/ffAGRNJru1FTz70BzhT3Zg",
  "expires_in": 3920,
  "scope": "https://www.googleapis.com/auth/drive.metadata.readonly https:",
  "token_type": "Bearer"
}
```

◆ Hướng dẫn viết mã



Xin lưu ý rằng có giới hạn về số lượng mã làm mới sẽ được cấp; một giới hạn cho mỗi tổ hợp ứng dụng/người dùng và một giới hạn khác cho mỗi người dùng trên tất cả các ứng dụng. Bạn nên lưu mã thông báo làm mới trong bộ nhớ dài hạn và tiếp tục sử dụng chúng miễn là chúng còn hiệu lực. Nếu ứng dụng của bạn yêu cầu quá nhiều mã làm mới, thì ứng dụng có thể gặp phải những giới hạn này. Trong trường hợp đó, các mã làm mới cũ hơn sẽ ngừng hoạt động.

Thu hồi mã thông báo

Trong một số trường hợp, người dùng có thể muốn thu hồi quyền truy cập đã cấp cho một ứng dụng. Người dùng có thể thu hồi quyền truy cập bằng cách truy cập vào phần Cài đặt tài khoản (<https://myaccount.google.com/permissions?hl=vi>). Hãy xem phần Xoá quyền truy cập của trang web hoặc ứng dụng (<https://support.google.com/accounts/answer/3466521?hl=vi#remove-access>) trong tài liệu hỗ trợ Các trang web và ứng dụng của bên thứ ba có quyền truy cập vào tài khoản của bạn để biết thêm thông tin.

Ứng dụng cũng có thể thu hồi quyền truy cập đã cấp cho ứng dụng đó theo phương thức có lập trình. Việc thu hồi theo chương trình là rất quan trọng trong trường hợp người dùng huỷ đăng ký, xoá một ứng dụng hoặc các tài nguyên API mà ứng dụng yêu cầu đã thay đổi đáng kể. Nói cách khác, một phần của quy trình xoá có thể bao gồm một yêu cầu API để đảm bảo các quyền đã cấp cho ứng dụng trước đó sẽ bị xoá.

DHP (#php)Python (#python)Ruby (#ruby)Node.js (#node.js)HTTP/REST

Để thu hồi mã thông báo theo phương thức lập trình, ứng dụng của bạn sẽ gửi yêu cầu đến `https://oauth2.googleapis.com/revoke` và đưa mã thông báo vào làm một tham số:

```
$ curl -d -X -POST --header "Content-type:application/x-www-form-urlencoded" \
https://oauth2.googleapis.com/revoke?token={token}
```

Mã thông báo có thể là mã thông báo truy cập hoặc mã thông báo làm mới. Nếu mã thông báo là mã truy cập và có mã làm mới tương ứng, thì mã làm mới cũng sẽ bị thu hồi.

Nếu yêu cầu thu hồi được xử lý thành công, thì mã trạng thái HTTP của phản hồi sẽ là **200**. Đối với các điều kiện lỗi, mã trạng thái HTTP **400** sẽ được trả về cùng với mã lỗi.

Điểm chính: Việc thu hồi sẽ xóa tất cả phạm vi OAuth 2.0 đã cấp cho một dự án, làm mất hiệu lực mọi mã truy cập hoặc mã làm mới đã phát hành cho tất cả ứng dụng được đăng ký trong dự án đó.

Lưu ý: Sau khi nhận được phản hồi thu hồi thành công, có thể phải mất một khoảng thời gian thì việc thu hồi mới có hiệu lực hoàn toàn.

Quyền truy cập dựa trên thời gian

Quyền truy cập dựa trên thời gian cho phép người dùng cấp cho ứng dụng của bạn quyền truy cập vào dữ liệu của họ trong một khoảng thời gian giới hạn để hoàn tất một hành động. Quyền truy cập có giới hạn thời gian được cung cấp trong một số sản phẩm của Google trong quy trình đồng ý, cho phép người dùng cấp quyền truy cập trong một khoảng thời gian giới hạn. Ví dụ: [Data Portability API](https://developers.google.com/data-portability?hl=vi) (<https://developers.google.com/data-portability?hl=vi>) cho phép chuyển dữ liệu một lần.

Khi người dùng cấp cho ứng dụng của bạn quyền truy cập dựa trên thời gian, mã làm mới sẽ hết hạn sau khoảng thời gian được chỉ định. Xin lưu ý rằng mã làm mới có thể bị vô hiệu hoá sớm hơn trong những trường hợp cụ thể; hãy xem [những trường hợp này](#).

(<https://developers.google.com/identity/protocols/oauth2?hl=vi#expiration>) để biết thông tin chi tiết. Trường `refresh_token_expires_in` được trả về trong phản hồi trao đổi mã uỷ quyền (https://developers.google.com/identity/protocols/oauth2/web-server?hl=vi#httprest_3) cho biết thời gian còn lại cho đến khi mã làm mới hết hạn trong những trường hợp như vậy.

Triển khai tính năng Bảo vệ nhiều tài khoản

Một bước bổ sung mà bạn nên thực hiện để bảo vệ tài khoản của người dùng là triển khai tính năng Bảo vệ trên nhiều tài khoản bằng cách sử dụng Dịch vụ bảo vệ trên nhiều tài khoản của Google. Dịch vụ này cho phép bạn đăng ký nhận thông báo về sự kiện bảo mật. Thông báo này cung cấp cho ứng dụng của bạn thông tin về những thay đổi lớn đối với tài khoản người dùng. Sau đó, bạn có thể sử dụng thông tin này để thực hiện hành động tùy thuộc vào cách bạn quyết định phản hồi các sự kiện.

Sau đây là một số ví dụ về các loại sự kiện mà Dịch vụ bảo vệ trên nhiều tài khoản của Google gửi đến ứng dụng của bạn:

- <https://schemas.openid.net/secevent/risc/event-type/sessions-revoked>
- <https://schemas.openid.net/secevent/oauth/event-type/token-revoked>
- <https://schemas.openid.net/secevent/risc/event-type/account-disabled>

Hãy xem trang Bảo vệ tài khoản người dùng bằng tính năng Bảo vệ nhiều tài khoản (<https://developers.google.com/identity/protocols/risc?hl=vi>) để biết thêm thông tin về cách triển khai tính năng Bảo vệ nhiều tài khoản và danh sách đầy đủ các sự kiện có sẵn.

Trừ phi có lưu ý khác, nội dung của trang này được cấp phép theo Giấy phép ghi nhận tác giả 4.0 của Creative Commons (<https://creativecommons.org/licenses/by/4.0/>) và các mẫu mã lập trình được cấp phép theo Giấy phép Apache 2.0 (<https://www.apache.org/licenses/LICENSE-2.0>). Để biết thông tin chi tiết, vui lòng tham khảo Chính sách trang web của Google Developers (<https://developers.google.com/site-policies?hl=vi>). Java là nhãn hiệu đã đăng ký của Oracle và/hoặc các đơn vị liên kết với Oracle.

Cập nhật lần gần đây nhất: 2025-12-19 UTC.